

Simulation d'Érosion Hydraulique et Thermique en Temps Réel sur GPU via Compute Shaders

César CHARBEY, Thomas BARASCUD, Jérémy NICOLAS, Lucas PAULO
Université de Montpellier
Projet de TER - Encadrant : Nicolas Lutz

Année Universitaire 2025-2026

Résumé

La génération de terrains virtuels réalistes nécessite souvent de simuler des phénomènes physiques complexes comme l'érosion. Ce document présente l'implémentation sur GPU (via Compute Shaders) du modèle d'érosion hydraulique rapide proposé par Mei et al. [1]. Afin de pallier le manque de détails géométriques inhérent à ces approches, tel que mis en évidence par Schott et al. [3], nous proposons une extension intégrant une simulation d'érosion thermique. Nos résultats préliminaires démontrent qu'il est possible d'augmenter le réalisme des pentes abruptes et la distribution des sédiments tout en maintenant des performances temps réel compatibles avec des applications interactives.

1 Introduction

Dans le domaine de l'informatique graphique, la création de mondes virtuels crédibles passe par une modélisation fidèle des paysages. Les mouvements d'eau, tels que la pluie et les rivières, sculptent ces terrains par des processus d'érosion, de transport de matière et de sédimentation. Si les méthodes basées sur la physique offrent le meilleur réalisme, elles souffrent historiquement de temps de calcul importants.

Ce projet vise à implémenter une solution d'érosion temps réel sur GPU basée sur les travaux de Mei et al. [1]. De plus, en réponse aux critiques récentes concernant le manque de précision et de détails de ces modèles classiques [3], nous intégrons une passe d'érosion thermique pour simuler l'éboulement gravitationnel de la roche et enrichir la topologie finale.

2 État de l'Art et Positionnement

La génération de terrains virtuels s'est historiquement appuyée sur des fonctions mathématiques de bruit (bruit de Perlin, fractales). Si ces méthodes garantissent une exécution instantanée, elles souffrent d'un manque inhérent de cohérence géomorphologique : les paysages générés sont dénués de réseaux de drainage ou de dépôts sédimentaires.

Pour pallier ce déficit, la recherche s'est tournée vers la simulation physique des processus d'érosion.

2.1 Érosion Hydraulique : Le Paradigme Eulérien

En 2007, Mei, Decaudin et Hu [1] ont provoqué un tournant paradigmatique en démontrant qu'il était possible de simuler l'érosion hydraulique à des fréquences d'affichage interactives sur GPU. Au lieu de résoudre les coûteuses équations tridimensionnelles de Navier-Stokes, leur modèle s'appuie sur une simplification bidimensionnelle : les équations de Saint-Venant (Shallow Water Equations).

Leur approche est de nature **Eulérienne** : l'espace est modélisé comme une grille discrète et immuable où chaque cellule met à jour son état physique (hauteur d'eau, vitesse, sédiments en suspension). L'écoulement hydrique y est calculé grâce au modèle des "tuyaux virtuels" (Virtual Pipe Model), accélérant l'eau via la différence de pression hydrostatique locale. Le transport de la masse sédimentaire est ensuite résolu par une advection semi-Lagrangienne rétrograde, une méthode inconditionnellement stable qui élimine les problèmes de divergence numérique (CFL).

2.2 Le Débat Conceptuel : Grilles vs Particules

L'état de l'art oppose fréquemment le paradigme Eulérien de Mei [1] à des approches **Lagrangiennes** (systèmes particuliers ou gouttelettes), récemment perfectionnées par Hartley et al. [4]. Si la méthode particulière excelle visuellement dans le tracé de micro-incisions organiques et de ravins profonds, elle se heurte à un obstacle matériel majeur sur GPU : les conditions de course (Race Conditions).

Dans un modèle Lagrangien, lorsque des milliers de particules d'eau convergent vers une même vallée, les fils d'exécution parallèles (threads) tentent d'écrire simultanément sur la même cellule de la grille topologique. La résolution de ces conflits impose l'utilisation de verrous atomiques (fonctions *InterlockedAdd*), ce qui complexifie la gestion de la concurrence et peut créer des goulots d'étranglement, là où l'approche Eulérienne garantit un accès mémoire parfaitement aligné et déterministe. Cela légitime son maintien

en tant que fondation algorithmique pour des simulations temps réel performantes.

2.3 La Révolution Matérielle des Compute Shaders

L'implémentation originelle de Mei [1] était fortement entravée par les contraintes matérielles de 2007. L'utilisation forcée des Fragment Shaders imposait un lourd détournement du pipeline de rasterisation (génération inutile de géométrie, paquetage des données physiques dans les canaux RGBA des textures, et redondance désastreuse des accès à la mémoire vidéo globale causant un goulot d'étranglement sévère).

Notre projet s'inscrit dans la transition architecturale vers les **Compute Shaders**. Cette technologie abstrait totalement le pipeline graphique. Les données sont structurées dans des textures avec accès direct via `imageLoad/imageStore`, offrant une flexibilité totale en lecture et écriture. Le double tamponnage (technique du "ping-pong") entre deux textures reste indispensable dans notre implémentation pour éviter les conditions de course (race conditions) lors de la mise à jour de la grille d'une frame à l'autre. Plus important encore, les Compute Shaders permettent d'exploiter la mémoire partagée (Shared Memory) des groupes de travail (Workgroups). En chargeant de manière coopérative une section de la carte des hauteurs dans ce cache ultra-rapide, nous minimisons drastiquement les requêtes redondantes vers la VRAM, transformant un algorithme historiquement limité par la bande passante en un modèle limité uniquement par la vitesse de calcul.

2.4 L'Érosion Thermique comme Nécessité Topologique

Bien que le modèle de Mei génère des écoulements fluides convaincants, il tend inexorablement à lisser les caractéristiques du terrain. Comme le critiquent vigoureusement des études récentes telles que celles de Schott et al. [3], ces approches manquent de précision géologique et nécessitent des mécanismes d'amplification de détails.

Pour répondre à cette critique, Jákó et Szirmay-Kalos [2] ont introduit la nécessité de coupler l'hydraulique avec un modèle rigoureux d'**érosion thermique**. L'eau a tendance à creuser des vallées très profondes aux parois parfaitement verticales, ce qui est physiquement instable. L'érosion thermique vient simuler l'effondrement gravitationnel et l'éboulement des matériaux rocheux dès lors que l'inclinaison locale dépasse l'angle de talus critique (angle de repos) du matériau.

2.5 Positionnement du Projet

L'objectif de notre projet de TER n'est donc pas la simple transposition d'un algorithme datant de 2007. Il s'agit de s'appuyer sur l'approche Eulérienne (pour sa robustesse matérielle) et sur la technologie des Compute

Shaders (pour sa bande passante optimisée), afin d'y adjoindre une passe de simulation d'érosion thermique moderne. Ce couplage dynamique vise à répondre directement aux enjeux actuels de la modélisation procédurale : générer en temps réel des paysages dotés de ruptures de pentes abruptes, de falaises réalistes et de dépôts sédimentaires organiques de haute fréquence, tout en préparant le terrain pour les futures avancées impliquant des strates volumétriques [5].

3 Méthodologie : Modèle de Base

3.1 Génération Topographique et Importation de Terrains

Le socle rocheux sur lequel opère notre simulation peut être défini de deux manières distinctes, sélectionnables dynamiquement via l'interface utilisateur (ImGui) lors de l'initialisation.

La première approche repose sur une génération procédurale exécutée intégralement sur le GPU. Pour sculpter ces massifs montagneux virtuels, nous combinons plusieurs octaves de bruit pseudo-aléatoire : un bruit de crêtes (*Ridged Noise*) est employé pour générer des chaînes de montagnes acérées et escarpées, tandis qu'un mouvement brownien fractionnaire (FBM - *Fractional Brownian Motion*) apporte les variations naturelles et les vallonnements du terrain de base.

La seconde approche intègre la prise en charge et le chargement de cartes de hauteurs réelles (*heightmaps*) importées sous forme d'images statiques. L'implémentation de cette fonctionnalité s'est révélée indispensable pour constituer notre référentiel d'évaluation. Elle permet en effet de mener des comparaisons rigoureuses et répétables entre nos différentes méthodes d'érosion (ainsi qu'avec l'implémentation CPU de référence) en garantissant une topologie d'entrée strictement identique.

3.2 Stabilité Numérique et Condition CFL

Un défi majeur des solveurs de fluides explicites, tel que le modèle des tuyaux virtuels, réside dans leur instabilité numérique si le pas de temps (Δt) est inadapté à la vitesse du fluide. Pour éviter les explosions mathématiques (où des hauteurs d'eau négatives apparaîtraient), nous avons implémenté une vérification dynamique de la condition de Courant-Friedrichs-Lewy (CFL). Le pas de temps doit strictement respecter l'équation : $\Delta t \leq \frac{\Delta x}{v_{max}}$. Notre système avertit l'utilisateur en temps réel via l'interface si l'accélération du fluide nécessite une réduction du pas de temps de simulation.

3.3 Implémentation du Pipeline et Gestion Mémoire

Bien que les fondements physiques de l'érosion hydraulique reposent sur le modèle Eulérien de Mei et al. [1], le

basculement vers une architecture Compute Shader moderne nécessite la mise en place de stratégies de synchronisation mémoire rigoureuses. Notre boucle de calcul est exécutée en parallèle sur la carte graphique (GPU) par des groupes de travail (*Workgroups*) de dimensions 16×16 *threads*.

Afin de prévenir les conditions de course (race conditions) intrinsèques aux calculs parallèles sur une grille topographique, nous avons structuré nos données selon une architecture de **double tamponnage** (*ping-pong buffering*). Les données d'état cruciales du terrain (Carte des hauteurs, Hauteur de l'eau, Quantité de sédiments) sontinstanciées en double.

À chaque itération de la physique, les Compute Shaders lisent l'état validé depuis le tampon de lecture (**READ**) et inscrivent les résultats mathématiques intermédiaires dans le tampon d'écriture (**WRITE**). Le pipeline itératif s'articule autour des passes suivantes :

1. **Ajout d'eau** (`water_increment.glsl`) : Application des précipitations (pluie) et des sources fluviales éventuelles sur la grille.
2. **Calcul des flux** (`flux_compute.glsl`) : Évaluation de la dynamique des fluides vers les quatre cellules voisines de von Neumann via le modèle des tuyaux virtuels (Pipe Model).
3. **Mise à jour des hauteurs et vitesses** (`water_update.glsl`) : Ajustement du niveau d'eau et dérivation des champs de vecteurs vitesse (advection spatiale).
4. **Érosion et Dépôt** (`erosion_deposition.glsl`) : Altération de la topologie en comparant la charge sédimentaire à la capacité de transport théorique de l'eau, dépendante de l'inclinaison locale et de la vitesse.
5. **Advection des sédiments** (`sediment_transport.glsl`) : Déplacement de la masse minérale en suspension via une approche semi-Lagrangienne rétrograde.
6. **Évaporation** (`evaporation.glsl`) : Retrait progressif du volume d'eau ambiant afin de réguler la masse hydrique globale du système.

L'intégrité temporelle de cette simulation est garantie par l'insertion de barrières de synchronisation explicites de l'API graphique (`glMemoryBarrier(GL_SHADER_IMAGE_ACCESS_BARRIER_BIT)`) entre chaque passe séquentielle. Ces directives forcent le GPU à purger ses caches et à finaliser toutes les écritures en attente avant d'autoriser la passe suivante à débiter la lecture des données. À l'issue du cycle complet d'érosion, les rôles des tampons (**READ** et **WRITE**) sont permutés pour l'itération ou le rendu visuel suivant.

4 Améliorations Proposées : Érosion Thermique

Afin de répondre au problème de manque de détails, nous avons développé un Compute Shader dédié à l'érosion

thermique.

4.1 Algorithme

Le processus est exécuté en parallèle sur l'ensemble de la grille via des groupes de travail (*Workgroups*) de dimensions 16×16 . Pour chaque cellule, l'algorithme évalue la différence de hauteur $d = h - h_{voisin}$ avec ses quatre voisins immédiats (voisinage de von Neumann). La stabilité de la roche est régie par la différence de hauteur maximale autorisée avant éboulement, calculée selon la géométrie de la grille : $\Delta h_{max} = \text{taille_cellule} \times \tan(\text{angle_talus})$.

Si la différence d excède ce seuil Δh_{max} (pente trop abrupte), la cellule cède de la matière à son voisin en contrebas. À l'inverse, si $d < -\Delta h_{max}$, la cellule réceptionne la matière d'un voisin surélevé. La quantité de matière transférée est proportionnelle à cet excédent, multipliée par une constante de vitesse d'érosion (`erosionRate`) et le pas de temps (Δt).

Afin de garantir la stabilité numérique et d'éviter un phénomène d'oscillation géométrique (où une cellule se viderait excessivement au profit de ses voisines), le transfert de matière est rigoureusement plafonné à 20 % de l'excès par voisin. La cellule pouvant transférer de la masse à ses quatre voisins simultanément, cette sécurité garantit une perte totale n'excédant jamais 80 % de l'excédent. Enfin, les cellules situées sur les bordures de la grille sont intentionnellement ignorées par la simulation, agissant comme des murs de contention statiques confinant le terrain.

4.2 Couplage avec l'Hydraulique

L'érosion thermique est intimement synchronisée avec la boucle de simulation hydraulique et s'exécute de manière systématique à chaque itération physique (qui peut s'exécuter plusieurs fois par image rendue à l'écran, selon le paramètre global `ITERATIONS_PER_FRAME`).

Dans l'architecture de notre pipeline de Compute Shaders, l'érosion thermique constitue la septième et ultime passe, intervenant immédiatement après l'étape d'évaporation de l'eau. Le shader lit la texture des hauteurs qui vient tout juste d'être altérée par l'érosion hydraulique (située dans le tampon d'écriture temporaire), calcule les éboulements nécessaires pour adoucir les ravins nouvellement formés, puis stocke le terrain stabilisé dans le tampon de lecture principal. Ce séquençage garantit non seulement que le rendu visuel affiche une géométrie cohérente, mais finalise également le mécanisme de "ping-pong" des textures, validant les données pour l'itération de simulation suivante.

5 Résultats et Évaluation

5.1 Analyse des Performances et Benchmarks

Afin de quantifier avec précision les gains octroyés par notre architecture GPU moderne, nous avons développé en

parallèle une implémentation rigoureusement équivalente s'exécutant intégralement sur le processeur central (CPU). Cette version CPU nous sert de référence (*baseline*) absolue pour calculer le ratio d'accélération matériel (*speedup*) et évaluer les performances globales de notre projet sur des cartes de hauteurs identiques.

Notre référentiel d'évaluation s'appuie également sur les données formellement documentées par Mei et al. en 2007 [1]. Lors de leurs expérimentations initiales (réalisées sur un processeur Pentium IV 2.40GHz équipé d'une carte graphique NVIDIA GeForce 8800 GTX), les auteurs mesuraient, pour une grille de résolution de 1024×1024 , un temps de simulation de 6.65 ms par passe physique et un temps de rendu visuel de 10.31 ms. Ces valeurs portaient le temps de cycle complet à 16.95 ms, correspondant à une fréquence d'environ 59 cycles par seconde (CPS).

Le tableau 1 présente les résultats de profilage comparatifs entre l'étude de 2007, notre implémentation CPU, et notre architecture GPU (Compute Shaders). Les temps d'exécution GPU ont été extraits via des requêtes asynchrones (*Timestamp Queries* OpenGL) afin d'isoler strictement la charge de calcul des latences processeur. Nos mesures ont été effectuées sur une configuration matérielle milieu de gamme moderne (AMD Radeon RX 5700, processeur AMD Ryzen 5 2600, résolution d'affichage 1440p).

Étape de calcul	Mei et al. (2007)	Notre CPU	Notre GPU
Simulation Physique	6.65 ms	726.215 ms	0.637 ms
Érosion Thermique	N/A	120.169 ms	0.033 ms
Rendu Visuel	10.31 ms	15.607 ms	0.453 ms
Temps de Cycle	16.95 ms	861.992 ms	1.123 ms

TABLE 1 – Tableau comparatif des performances (Résolution : 1024×1024). Les données de Mei et al. [1] servent de référence historique. Les tests modernes utilisent un Ryzen 5 2600 et une RX 5700.

Les résultats obtenus confirment de manière spectaculaire la supériorité de l'architecture GPU pour la résolution de modèles Eulériens sur grilles. L'implémentation CPU séquentielle s'avère totalement inadaptée au temps réel, affichant un temps de cycle critique de près de 862 ms (soit à peine 1,16 image par seconde). À l'inverse, l'implémentation sur Compute Shaders exécute le même cycle en 1,12 ms, atteignant une fréquence interactive de l'ordre de 890 FPS. Cela représente un facteur d'accélération global impressionnant de plus de **767x** par rapport au CPU.

Cette accélération est particulièrement notable sur la passe d'érosion thermique. Reposant sur des calculs purement locaux (voisinage de von Neumann à 4 cellules), cet algorithme est "naturellement parallélisable" (*embarrassingly parallel*). Le passage d'une exécution séquentielle CPU (120,16 ms) à une exécution massivement parallèle GPU via mémoire partagée (0,033 ms) illustre un gain de performance supérieur à un facteur 3600. Le coût de notre amélioration thermique s'avère ainsi parfaitement négligeable sur GPU par rapport au gain géomorphologique structurel qu'elle apporte.

Enfin, il est crucial de noter que notre architecture lo-

gicielle permet de dissocier la fréquence de la simulation physique de la fréquence de rendu visuel (découplage). Étant donné la vaste réserve de puissance de calcul matérielle libérée (framerate effectif d'environ 900 FPS), il est possible d'exécuter plusieurs dizaines d'itérations physiques complètes entre chaque rafraîchissement d'écran (*ITERATIONS_PER_FRAME*), offrant la capacité d'accélérer artificiellement l'écoulement du temps géologique tout en conservant une fluidité interactive ininterrompue lors de la navigation.

5.2 Qualité Visuelle et Interactivité

L'ajout de l'érosion thermique permet de briser les arêtes non naturelles créées par le creusement hydraulique, résultant en des formations de type éboulis beaucoup plus proches de la réalité. Par ailleurs, notre simulateur agit comme un véritable outil de conception en temps réel : grâce à une interface graphique dédiée, l'utilisateur peut ajuster à la volée les constantes géologiques ou introduire dynamiquement des sources fluviales pour observer l'adaptation organique du réseau de drainage.

6 Conclusion et Perspectives

L'intégration de l'érosion thermique au sein d'une boucle d'érosion hydraulique moderne via Compute Shaders permet de répondre efficacement au manque de détails reproché aux simulations temps réel. Les prochaines étapes consisteront à finaliser nos comparatifs de performances (*benchmarks*) face à la version CPU, et à évaluer rigoureusement le comportement de notre algorithme sur des *heightmaps* réelles.

Références

- [1] X. Mei, P. Decaudin, and B. Hu. *Fast hydraulic erosion simulation and visualization on gpu*. In 15th Pacific Conference on Computer Graphics and Applications (PG'07), pages 47-56. IEEE, 2007.
- [2] B. Jákó and L. Szirmay-Kalos. *Fast Hydraulic and Thermal Erosion on the GPU*. In Eurographics (Short Papers), pages 57-60, 2011.
- [3] H. Schott, E. Galin, E. Guérin, A. Paris, and A. Peytavie. *Terrain amplification using multi scale erosion*. ACM Transactions on Graphics (TOG), 43(4) :1-12, 2024.
- [4] P. Hartley, N. Mellado, C. Fiorio, and N. Faraj. *Flexible terrain erosion*. Computer Graphics Forum, 43(2), 2024.
- [5] S. Nilles, J. Günther, M. Wagner, and C. Müller. *3D Real-Time Hydraulic Erosion Simulation using Multi-Layered Heightmaps*. In Vision, Modeling, and Visualization (VMV), 2024.